

TY B.Tech. – ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

PRACTICAL ASSIGNMENT

1.Student Details

Student Name	Pankaj Shinde
PRN	202401100005
Branch	Software Engineering
Division	A
Batch	T4
Roll No.	SE-A7
Date of Submission	28 /8/2026

2. Practical Details

Practical No.	1.1
Practical Title	PEAS Descriptors of Task Environment
Date Performed	13-08-2026
Faculty Name	Dr. Khushal Khairnar

3. Problem Statement :

Demonstrate PEAS Descriptors of Task Environment using Vacuum Cleaner Intelligent Agent Application.

4. Objectives

1. To understand the concept of intelligent agents and represent a task environment using the PEAS (Performance Measure, Environment, Actuators, Sensors) framework.
2. To implement a simple Vacuum Cleaner Intelligent Agent using Prolog and demonstrate its decision-making based on the current environment.
3. To develop a basic Expert System in Prolog using facts and rules to provide conclusions for a specialized knowledge domain.

5.Expected Learning Outcomes

After completing this practical, the student will be able to:

Identify and describe the **PEAS components** of an intelligent agent and understand different types of task environments.

- Implement an intelligent agent using **Prolog facts, rules, predicates, and logical inference**.
- Develop a simple **rule-based Expert System** and use it to derive conclusions from a given knowledge base.

6. Theory / Concept

An **intelligent agent** is a system that perceives its environment through sensors and performs actions through actuators to achieve a specific goal. The agent observes the current state of the environment and selects an appropriate action based on predefined rules or available knowledge.

PEAS is a framework used to describe the task environment of an intelligent agent. It consists of:

- **P – Performance Measure:** Specifies how the performance and success of the agent are evaluated.
- **E – Environment:** Defines the surroundings in which the agent operates.
- **A – Actuators:** Specifies the mechanisms through which the agent performs actions.
- **S – Sensors:** Specifies how the agent perceives and obtains information from the environment.

Prolog is a logic programming language widely used in Artificial Intelligence for representing knowledge and performing logical reasoning. A Prolog program mainly consists of:

- **Facts:** Statements or information that are considered true.
- **Rules:** Logical conditions used to derive new information from existing facts.
- **Queries:** Questions given to the knowledge base to obtain conclusions.

An **Expert System** is an AI-based system that uses a **knowledge base of facts and rules** along with an **inference mechanism** to solve problems or make decisions in a specific domain. It applies logical rules to the available facts and produces appropriate conclusions.

In this practical, Prolog is used to implement a **Vacuum Cleaner Intelligent Agent** and demonstrate its decision-making using the PEAS framework. A rule-based **Expert System** is also developed using facts and rules to demonstrate knowledge representation and logical inference.

7. Algorithm / Pseudocode

Write the step-by-step algorithm or pseudocode for the proposed solution.

Step 1: Start

Step 2: Define the rooms, their adjacency relationships, and initialize all rooms as dirty. Set the vacuum cleaner's initial location to room A.

Step 3: Check the current location of the vacuum cleaner and determine whether the current room is dirty.

Step 4: If the current room is dirty, clean the room and update its status to clean.

Step 5: If the current room is clean, move the vacuum cleaner to an adjacent dirty room.

Step 6: Repeat the cleaning and movement process until no dirty rooms remain. Then stop the vacuum cleaner and display the completion message.

Step 7: Stop

8. Technology / Tools / Libraries Used

Technology / Tool	Purpose
Prolog	Used to implement the rule-based logic and behavior of the Vacuum Cleaner Intelligent Agent.
SWI-Prolog	Used to write, compile, and execute the Prolog program.
Libraries / Packages	No external libraries or packages were required for this basic implementation.
Dataset / Other Resources	No external dataset was required.

09. Implementation / GitHub Repository

Particular	Details
GitHub Repository URL	https://github.com/Pankaj-Shinde-07/MDM_AI-ML_AS01.git
Repository Name	MDM_AI-ML_AS01
Branch Used	main
Commit ID / Latest Commit	715d0fc
Date of Last Commit	28/8/2026

10. Test Cases / Input and Output

Test Case	Input	Expected Output	Actual Output	Status
Clean current room	Vacuum in a, dirty(a)	Clean room a	Cleaning room a...	Pass
Move to dirty room	Vacuum in a, dirty(b)	Move from a to b	Moving from room a to room b...	Pass
Clean Room B	Vacuum in b, dirty(b)	Clean room b	Cleaning room b...	Pass
Move to Room C	Vacuum in c, dirty(c)	Clean room c	Cleaning room c...	Pass

For ML practicals, include appropriate performance metrics, graphs, tables, or model outputs wherever applicable.

11. Results / Observations

- Mention the important results obtained during execution.
- The vacuum-cleaning agent successfully identified dirty rooms and performed cleaning actions.
- The vacuum initially started in **Room A** and cleaned it.
- The agent then moved between adjacent rooms and cleaned **Rooms B and C**.
- The dirty/1 facts were dynamically removed using retract/1 after cleaning.
- Once no dirty rooms remained, the stop action was selected and the program terminated successfully.

- The execution demonstrated the use of **dynamic facts, rules, recursion, and state changes in Prolog**.

Screenshots / Graphs / Output:

```
35 ?- ["D:/prolog/vaccum.pl"].
true.

36 ?- start.
Cleaning room a...
Moving from room a to room b...
Cleaning room b...
Moving from room b to room c...
Cleaning room c...
All rooms are clean. Stopping...
1 All rooms are clean. Stopping...
true ▲
```

12. Performance Evaluation / Analysis

- **Correctness:** The agent correctly cleaned all three rooms and stopped after the environment became clean.
- **Number of rooms cleaned:** 3
- **Cleaning actions:** 3
- **Movement actions:** 2
- **Termination:** Successful, after all dirty/1 facts were removed.
- **Search efficiency:** The simple environment required only a small number of actions because the rooms were arranged linearly as A–B–C.

13. Conclusion

Write a brief conclusion describing what was implemented and what was learned. The vacuum cleaner agent was successfully implemented in Prolog using facts, rules, dynamic predicates, and recursion. The agent was able to identify dirty rooms, move between adjacent rooms, clean them, update the environment, and stop when all rooms were clean. This practical helped in understanding how Prolog can be used to represent an intelligent agent and demonstrated concepts such as logical inference, dynamic state modification, recursion, and backtracking.

14. Assignment Questions

Q1. What are the different actions performed by the vacuum cleaner agent?

Answer:

The vacuum cleaner agent can perform three actions: clean, move, and stop. It cleans the current room if it is dirty, moves to an adjacent dirty room, and stops when all rooms are clean.

Q2. How are the room states updated in the vacuum cleaner program?

Answer:

The retract/1 predicate is used to remove the dirty fact after a room is cleaned. The vacuum's location is updated using retract/1 to remove the old location and assertz/1 to add the new location.

15. Implementation / Source Code

```
:- dynamic dirty/1.
```

```
:- dynamic vacuum_location/1.
```

```
room(a).
```

```
room(b).
```

```
room(c).
```

```
adjacent(a, b).
```

```
adjacent(b, a).
```

```
adjacent(b, c).
```

```
adjacent(c, b).
```

```
dirty(a).
```

```
dirty(b).
```

dirty(c).

vacuum_location(a).

action(clean) :-

vacuum_location(Room),

dirty(Room).

action(move(ToRoom)) :-

vacuum_location(CurrentRoom),

adjacent(CurrentRoom, ToRoom),

dirty(ToRoom).

action(stop) :-

\+ dirty(_).

perform(clean) :-

vacuum_location(Room),

dirty(Room),

retract(dirty(Room)),

format("Cleaning room ~w...~n", [Room]).

perform(move(ToRoom)) :-

vacuum_location(CurrentRoom),

retract(vacuum_location(CurrentRoom)),

assertz(vacuum_location(ToRoom)),

format("Moving from room ~w to room ~w...~n", [CurrentRoom, ToRoom]).

perform(stop) :-

```
format("All rooms are clean. Stopping...~n", []).
```

start :-

```
action(Action),
```

```
perform(Action),
```

```
Action \= stop,
```

```
start.
```

start :-

```
action(stop),
```

```
perform(stop).
```

16. Student Declaration

I hereby declare that the above practical work has been completed by me and that the implementation, analysis, and answers are my own work.

Student Name		PRN	Signature
Pankaj	Shinde	202401100005	